Bank Management Database System

Project Phase 2 Report: Relational Mapping & SQL Implementation

Borhan Javadian - 35012
Bardiya Shavandi - 33588
Faraz Sahabi - 34557

1 Relational Schema

The following tables represent the conversion of the Phase 1 ER diagram into a relational model. Primary keys (<u>PK</u>) and Foreign keys (*FK*) are identified for each table.

1.1 Management & Workforce

- **Manager**: <u>ManagerID</u>, FullName.
- **Employee**: <u>EmployeeID</u>, FullName, Position, Salary.
- **Manages**: <u>ManagerID (FK)</u>, <u>EmployeeID (FK)</u>, <u>From</u>, Until.
- **WorksIn**: <u>EmployeeID (FK)</u>, <u>BranchID (FK)</u>, <u>From</u>, Until.

1.2 Customer & Core Banking

- **Customer**: <u>CustomerID</u>, FullName, Phone, Email.
- **Account**: <u>AccountID</u>, Number, Balance, Open (Date), CustomerID (FK), BranchID (FK).
- **Branch**: <u>BranchID</u>, Address, BankID (FK).

1.3 Institutional & Global Structure

- **Bank**: <u>BankID</u>, Name.
- **Country**: <u>CountryID</u>, Name.
- **PlacedIn**: <u>BankID (FK)</u>, <u>CountryID (FK)</u>.

1.4 Transactions & Subtypes

- **Transaction**: <u>TransactionID</u>, Amount, Currency, Date, AccountID (FK).
- **Transfer**: <u>TransactionID (FK)</u>, Recipient.
- **Loan**: <u>TransactionID (FK)</u>, Maturity.

2 Mapping Explanation

The transition from ER diagram to relational tables followed standard mapping rules with specific logic for historical tracking:

1. **Entity Mapping:** Core entities (Bank, Customer, etc.) were converted into base tables. Subtypes (*Transfer, Loan*) use the Primary Key of the parent *Transaction* table as their own PK and FK (ISA mapping).
2. **1:M Relationships (Foreign Keys):** Relationships such as *Owns*, *BelongsTo*, and *OwnedBy* were mapped by placing the "One" side's Primary Key as a Foreign Key in the "Many" side table (e.g., BankID in Branch).
3. **Relationship Tables with History:** Although *Manages* and *WorksIn* are 1:M, they were mapped to separate tables to handle the attributes `From` and `Until`. To allow an employee to have a history of different stints, the Primary Keys are composite: `(ManagerID, EmployeeID, From)` and `(EmployeeID, BranchID, From)`.
4. **M:M Relationship:** The *PlacedIn* relationship between Bank and Country was mapped to a junction table with a composite primary key consisting of both entities' IDs.

3 Integrity Constraints

- **Not Null:** All primary keys and mandatory attributes (Names, Dates, Amounts) are set to `NOT NULL`. `Phone` and `Email` remain optional.
- **Referential Integrity:**
  - **RESTRICT:** Used for core tables (*Bank, Branch, Account, Country*) to prevent deletion of entities that are currently linked to other records.
  - **CASCADE:** Used for relationship history (*Manages, WorksIn*) and ISA subtypes (*Transfer, Loan*) so that if an Employee or Transaction is deleted, their history/details are removed automatically.
- **Check Constraints:** `Salary` must be > 0 and `Balance` must be ≥ 0.

4 SQL CREATE TABLE Statements

```sql
-- 1. Institutional & Global Structure
CREATE TABLE Country (
    CountryID INT PRIMARY KEY,
    Name VARCHAR(255) NOT NULL
);

CREATE TABLE Bank (
    BankID INT PRIMARY KEY,
    Name VARCHAR(255) NOT NULL
);

CREATE TABLE PlacedIn (
    BankID INT NOT NULL,
    CountryID INT NOT NULL,
    PRIMARY KEY (BankID, CountryID),
    FOREIGN KEY (BankID) REFERENCES Bank(BankID) ON DELETE RESTRICT ON UPDATE CASCADE,
    FOREIGN KEY (CountryID) REFERENCES Country(CountryID) ON DELETE RESTRICT ON UPDATE CASCADE
);

-- 2. Management & Workforce
CREATE TABLE Manager (
    ManagerID INT PRIMARY KEY,
    FullName VARCHAR(255) NOT NULL
);

CREATE TABLE Employee (
    EmployeeID INT PRIMARY KEY,
    FullName VARCHAR(255) NOT NULL,
    Position VARCHAR(100) NOT NULL,
    Salary DECIMAL(15, 2) NOT NULL CHECK (Salary > 0)
);

CREATE TABLE Manages (
    ManagerID INT NOT NULL,
    EmployeeID INT NOT NULL,
    StartFrom DATE NOT NULL,
    Until DATE,
    PRIMARY KEY (ManagerID, EmployeeID, StartFrom),
    FOREIGN KEY (ManagerID) REFERENCES Manager(ManagerID) ON DELETE RESTRICT ON UPDATE CASCADE,
    FOREIGN KEY (EmployeeID) REFERENCES Employee(EmployeeID) ON DELETE CASCADE ON UPDATE CASCADE
);

-- 3. Core Banking
CREATE TABLE Branch (
    BranchID INT PRIMARY KEY,
```

```sql
    Address VARCHAR(255) NOT NULL,
    BankID INT NOT NULL,
    FOREIGN KEY (BankID) REFERENCES Bank(BankID) ON DELETE
       ↪ RESTRICT ON UPDATE CASCADE
);

CREATE TABLE WorksIn (
    EmployeeID INT NOT NULL,
    BranchID INT NOT NULL,
    StartFrom DATE NOT NULL,
    Until DATE,
    PRIMARY KEY (EmployeeID, BranchID, StartFrom),
    FOREIGN KEY (EmployeeID) REFERENCES Employee(EmployeeID) ON
       ↪ DELETE CASCADE ON UPDATE CASCADE,
    FOREIGN KEY (BranchID) REFERENCES Branch(BranchID) ON DELETE
       ↪ RESTRICT ON UPDATE CASCADE
);

CREATE TABLE Customer (
    CustomerID INT PRIMARY KEY,
    FullName VARCHAR(255) NOT NULL,
    Phone VARCHAR(20), -- String to allow leading zeros
    Email VARCHAR(255)
);

CREATE TABLE Account (
    AccountID INT PRIMARY KEY,
    Number VARCHAR(50) NOT NULL UNIQUE,
    Balance DECIMAL(15, 2) NOT NULL CHECK (Balance >= 0),
    OpenDate DATE NOT NULL,
    CustomerID INT NOT NULL,
    BranchID INT NOT NULL,
    FOREIGN KEY (CustomerID) REFERENCES Customer(CustomerID) ON
       ↪ DELETE RESTRICT ON UPDATE CASCADE,
    FOREIGN KEY (BranchID) REFERENCES Branch(BranchID) ON DELETE
       ↪ RESTRICT ON UPDATE CASCADE
);

-- 4. Transactions
CREATE TABLE Transaction (
    TransactionID INT PRIMARY KEY,
    Amount DECIMAL(15, 2) NOT NULL CHECK (Amount > 0),
    Currency VARCHAR(10) NOT NULL,
    TransDate DATE NOT NULL,
    AccountID INT NOT NULL,
    FOREIGN KEY (AccountID) REFERENCES Account(AccountID) ON
       ↪ DELETE RESTRICT ON UPDATE CASCADE
);

CREATE TABLE Transfer (
    TransactionID INT PRIMARY KEY,
```

```sql
    Recipient VARCHAR(255) NOT NULL,
    FOREIGN KEY (TransactionID) REFERENCES Transaction(
       ↪ TransactionID) ON DELETE CASCADE ON UPDATE CASCADE
);

CREATE TABLE Loan (
    TransactionID INT PRIMARY KEY,
    Maturity DATE NOT NULL,
    FOREIGN KEY (TransactionID) REFERENCES Transaction(
       ↪ TransactionID) ON DELETE CASCADE ON UPDATE CASCADE
);
```

5 Appendix - AI Usage Documentation

5.1 Conversation Link

The full transcript of the iterative design process, including the logic used to derive the final relational schema and SQL constraints, can be accessed at the following link:

- **Chat Link:** `https://gemini.google.com/share/18ec4e33c46a`